PE-JV-003

Aula 03 - Banco de Dados em Larga Escala (PostgreSQL & NoSQL)

Aquecimento

Kahoot

Revisão da Aula 02: Resiliência, Circuit Breaker, Rate Limiter e Idempotência.
Aguarde o link do professor.

O Desafio de Engenharia

Na última aula, blindamos a API de Checkout contra lentidões e falhas externas. Sucesso! Nossa API agora aceita **alta concorrência de acessos simultâneos**.

O problema:
Agora o gargalo mudou de lugar. Esse alto volume de requisições está tentando gravar os pedidos em nossa única instância de **PostgreSQL**. As transações estão enfileirando, o banco travou (*Lock*), e o CPU bateu 100%.

Como escalamos nosso armazenamento para suportar o tráfego da Black Friday sem perder a consistência dos pagamentos?

A Missão do Dia

No seu documento de System Design (Aula 03):

- Definir a estratégia de Chave Primária para sistemas distribuídos (Sequence, UUIDv4 ou UUIDv7).
- Desenhar a estratégia de armazenamento: quando otimizar o PostgreSQL, quando adotar NoSQL ou como combiná-los em Persistência Poliglota.
- Definir a fronteira de consistência: onde o rigor ACID é inegociável e onde a Consistência Eventual é aceitável.
- Estabelecer a estratégia de concorrência com Locks e investigar gargalos de queries usando `EXPLAIN`.

Sala de Aula Invertida: A Pesquisa

Acesse os materiais compartilhados e pesquise:

1. **A Guerra dos IDs:** Por que o *Auto-Increment* vira gargalo distribuído? Qual o custo do **UUIDv4** no índice B-Tree e o que o **UUIDv7** resolve?
2. **Concorrência e Travas:** Qual a diferença entre **Lock Otimista (** `@Version` **)** e **Lock Pessimista (** `SELECT FOR UPDATE` **)**? Quando cada um gera exceções ou enfileira conexões?
3. **Além do MongoDB:** Qual o ponto ótimo entre Bancos Relacionais, Documentais e Key-Value? Quando usar Persistência Poliglota?
4. **A Batalha da Consistência:** Quando faz sentido abrir mão do rigor **ACID** para usar **Consistência Eventual** (BASE)?
5. **Olimpíadas do SQL (PostgreSQL):** Como o índice funciona e por que ele degrada o `INSERT`? Como o `EXPLAIN ANALYZE` e ferramentas visuais (Dalibo) ajudam a achar gargalos?

O Desafio Arquitetural

No seu documento de System Design, estruture suas decisões técnicas:

1. **Armazenamento & NoSQL:** Qual motor utilizar para o Checkout? Além do modelo documental, quais outras famílias NoSQL fariam sentido para o ecossistema e para quais casos de uso?
2. **Dualidade de Concorrência:**
    - Qual abordagem de lock usar para **Status do Pagamento** vs **Estoque**?
    - Quais os riscos de cada abordagem e como mitigá-los?
3. **Troubleshooting & Decisão de Índices:**
    - Qual o passo a passo para diagnosticar e comprovar a causa raiz de queries lentas?
    - Um dev propôs indexar todas as colunas da tabela: você aprovaria esse PR? Por quê?

Debate

É hora de debater com o comitê de engenharia (Staff Engineer):

- **Estratégias de PK:** Como decidir entre Sequence, UUIDv4 ou UUIDv7 para desacoplar a geração de chaves?
- **Lock Otimista e Pessimista:** Como equilibrar abortos no commit (`OptimisticLockException`) versus filas de conexões e deadlocks?
- **O Mito do "Indexa Tudo":** Como provar o impacto do índice no plano de execução e na vazão de escrita?
- **Critérios de Escolha (SQL, NoSQL ou Ambos):** Para quais casos de uso cada tecnologia faz sentido e quais justificativas técnicas embasam a escolha de uma em relação à outra?

Projeto Individual (System Design)

Cenário de Engenharia (Checkout na Black Friday):

- **Armazenamento & PK:** Transação financeira exige ACID; catálogo exige alta vazão. Como gerar IDs distribuídos sem colisão ou gargalo?
- **Concorrência:** Mudança de status do pedido (baixa contenção) vs Reserva de estoque (alta disputa simultânea pelo mesmo SKU).
- **Incidente Real:** Latência p99 explodiu para 4.8s e pool HikariCP esgotou por buscas lentas em pedidos sem índice.

Sua Missão:

- Responda às decisões do **Desafio Arquitetural** aplicadas diretamente ao cenário acima.
- Utilize o **Template da Aula 03** (disponível no LMS) e versione a entrega no repositório do seu projeto.

Prática de Código (Opcional)

Disponibilizamos um artefato auxiliar no LMS contendo um desafio prático de código. O objetivo é comparar a performance de inserção e consulta no PostgreSQL usando IDs Sequenciais versus UUIDv4 e analisar o plano de execução com `EXPLAIN ANALYZE`.

Próxima Aula: Microsserviços e Transações Distribuídas

Agora nosso armazenamento aguenta a carga. Mas se separarmos o sistema em microsserviços, como garantir consistência entre múltiplos bancos?

Tópicos:

- A Queda do ACID: Como lidar com transações distribuídas entre serviços.
- O Padrão Saga: Coreografia vs Orquestração e Transações Compensatórias.
- O Perigo do Dual Write: Atomicidade com Transactional Outbox (CDC/Debezium).
- Desacoplando Leitura e Escrita com CQRS.

Referências para Leitura Prévia:

- 📄 Pattern: Saga (Chris Richardson)
- 📄 Saga Pattern (Martin Fowler)
- 📄 Transactional Outbox (Microservices.io)
- 📄 CQRS (Martin Fowler)

Referências da Aula 03 (Banco de Dados)

Links oficiais para estudo e aprofundamento:

- 📄 **Chaves Primárias & B-Trees:** Goodbye integers, hello UUIDv7 (Buildkite)
- 📄 **Estrutura de Índices:** Anatomy of an Index (UseTheIndexLuke)
- 📄 **Scan Types no EXPLAIN:** Postgres Scan Types in Explain Plans (Crunchy Data)
- 🛠️ **Visualizador de Planos:** explain.dalibo.com (PEV2)
- 📄 **Locks & Concorrência:** Guia de Locks e Lost Update (Vlad Mihalcea)
- 📄 **Teorema CAP & NoSQL:** O que é o Teorema CAP? (IBM) e SQL vs NoSQL (IBM)
- 🗺️ **Árvore de Decisão de Banco:** Data Store Decision Tree (Microsoft Learn)
- 📄 **Persistência Poliglota:** Polyglot Persistence (Martin Fowler)
- 🎥 **Vídeo Clássico:** Introduction to NoSQL - GOTO 2012 (Martin Fowler)

Obrig.ADA